Formalisation Mathématique du Modèle Oracle

(abstraction programme/automates, obligations locales et correction)

Projet Kairos

7 mars 2026

1 Objectif

Cette note formalise un schéma de preuve pour un programme synchrone à état, décrit par un automate de contrôle S couplé à une mémoire M. Un pas d'exécution lit une entrée courante $i \in I$, part d'un état de contrôle $s \in S$ et d'une mémoire courante $m \in M$, puis produit un état de contrôle suivant $s' \in S$, une mémoire suivante $m' \in M$ et une sortie $o \in O$. On note ce pas local

$$(s, m, i) \longmapsto (s', m', o).$$

Il est réalisé par une transition de l'automate de contrôle, munie d'une garde, d'une mise à jour mémoire et d'une fonction de sortie.

Cette dynamique locale est ensuite itérée sur un flux d'entrée $u \in \mathrm{Str}(I)$, noté

$$u = (i_0, i_1, \dots),$$

ce qui induit simultanément une suite de configurations

$$(s_0, m_0) \to (s_1, m_1) \to (s_2, m_2) \to \cdots$$

et un flux de sortie

$$y = (o_0, o_1, \dots) \in \mathrm{Str}(O),$$

d'où la trace observable entrée/sortie

$$\mathrm{run}(u) = ((i_0, o_0), (i_1, o_1), \dots) \in \mathrm{Str}(I \times O).$$

La spécification externe est donnée par deux automates de sûreté : un automate d'hypothèse A, interprété sur les entrées, et un automate de garantie G, interprété sur la trace $\mathrm{run}(u)$. Le cadre est déterministe : au tick courant, le programme sélectionne une transition unique, et chaque automate (A et G) sélectionne une arête unique pour la paire (état courant, observation courante).

Les obligations générées sont de deux natures :

— **obligations issues du produit** programme × automates (`Compatibility`/`Instrumentation` selon la classification), construites à partir des pas produits bien formés et de leurs cibles (sûres ou dangereuses) ;

— **obligations de contrat local** déjà attachées aux transitions (`UserContract`, `Coherency`, etc., selon la provenance).

Le mécanisme de preuve est local par transition (lecture Hoare) : chaque obligation est évaluée sur un contexte de pas, puis propagée le long des exécutions. L'oracle est supposé certifier la validité de ces obligations locales. Le résultat visé est donc explicitement *conditionnel* : si les obligations/contrats locaux générés sur les transitions sont validés, alors la garantie globale de sûreté est satisfaite sous l'hypothèse d'entrée.

Formellement, on établit :

$$\forall u \in \mathrm{Str}(I), \quad \mathrm{AvoidA}(u) \Rightarrow \mathrm{AvoidG}(\mathrm{run}(u)),$$

où AvoidA et AvoidG signifient "n'atteint jamais l'état *bad*" dans les automates de sûreté correspondants.

2 Plan du document

Le document est organisé en quatre niveaux de lecture, du plus intuitif au plus formel :

1. **Vue d'ensemble par exemples.** On présente d'abord `delay_int` et `toggle` comme deux fils rouges pour donner l'intuition du programme, des automates A et G, du produit $programme \times A \times G$, puis des obligations locales.
2. **Modèle formel.** On fixe ensuite les objets mathématiques : automate de programme, flux, automates de sûreté, sémantique du produit et pas réalisés.
3. **Chaîne de preuve.** On définit formellement les obligations locales, leur génération à partir des pas dangereux, puis on énonce la chaîne de preuve "violation globale $\Rightarrow$ pas dangereux local $\Rightarrow$ obligation générée violée".
4. **Lien avec Rocq et l'implémentation.** On termine par la traçabilité vers les modules Rocq et par l'explication du rôle du backend OCaml actuel.

La section suivante regroupe uniquement les conventions de lecture. Le premier vrai point d'entrée pédagogique est la section "Vue d'ensemble par exemples".

3 Préliminaires : notations et conventions

3.1 Conventions de nommage

— **Domaines/types** (majuscules) : $I, O, M, S, Q, \ldots$

— **Éléments** (minuscules) : $i \in I,\ o \in O,\ m \in M,\ s \in S,\ q \in Q$

— **Flux** (lettres u, w) : $u \in \mathrm{Str}(I)$, $w \in \mathrm{Str}(I \times O)$

— **Transitions/arêtes** : t pour le programme, e_A, e_G pour automates

— **Fonctions de transition** : δ_A, δ_G pour automates ; step pour programme

— **Indices temporels** : $k, j, n \in \mathbb{N}$, avec k "tick courant", $k+1$ "tick suivant"

— **États initiaux** : indice 0, notés s_0, m_0, q_0

— **États mauvais** : notés bad_A, bad_G

3.2 Notations mathématiques globales

— $\mathrm{Str}(X) := \mathbb{N} \to X$ (flux infini de valeurs de X)

— $\mathrm{run}(u)$: trace observable entrée/sortie produite par le programme sous u

— $\mathrm{AvoidA}(u)$: "A n'atteint jamais bad_A sur u"

— $\mathrm{AvoidG}(\mathrm{run}(u))$: "G n'atteint jamais bad_G sur $\mathrm{run}(u)$"

— $\mathrm{cfg}(u, k)$: configuration interne du programme au tick k

— $\mathrm{rps}(u, k)$: état du produit au tick k

— $\mathrm{BadLocal}(u, k)$: existence d'un pas local dangereux au tick k

— $\mathrm{GeneratedBy}(o, t, \varphi)$: φ est générée (origine o, transition t)

3.3 Conventions de produits et projections

Pour un tuple (a, b, c), on utilise des fonctions de projection nommées par domaine (par exemple composante programme, composante automate A, composante automate G). Dans les définitions formelles, ces projections sont toujours explicites afin d'éviter les ambiguïtés entre "état courant" et "état cible" d'un pas.

3.4 Conventions logiques

— Toutes les obligations locales sont des prédicats booléens sur un contexte de pas.

— "Valide" signifie : vrai pour tout tick, sur toute exécution.

— Les implications "oracle correct" sont lues dans le sens soundness :

$$\text{Oracle}(\varphi) = \mathsf{true} \Rightarrow \varphi \text{ valide.}$$

— Les preuves par contradiction utilisent explicitement :

$$\neg\forall k\, P(k) \Rightarrow \exists k\, \neg P(k)$$

sur $\mathbb{N}$.

4 Vue d'ensemble par exemples

Cette section donne une vue opérationnelle de la chaîne complète, en parallèle sur `delay_int` et `toggle`. Elle doit se lire comme une visite guidée : on y introduit les objets dans l'ordre où ils interviennent réellement dans le raisonnement, sans encore chercher la généralité maximale.

4.1 Étape 1 — Programme → automate de contrôle

delay_int. On considère un automate à deux états :

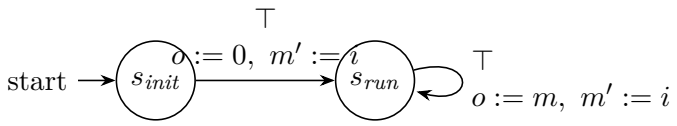
$$s_{init} \xrightarrow{\top \;/\; o{:=}0,\; m'{:=}i} s_{run}, \qquad s_{run} \xrightarrow{\top \;/\; o{:=}m,\; m'{:=}i} s_{run}.$$

toggle. Le programme alterne 0/1 en sortie, avec deux états de contrôle :

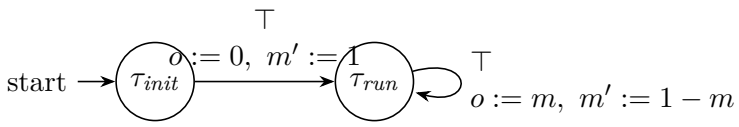
$$t_{init} :\ \tau_{init} \to \tau_{run},\ o := 0,\ m' := 1,$$

$$t_{run} :\ \tau_{run} \to \tau_{run},\ o := m,\ m' := 1 - m.$$

delay_int



toggle



4.2 Étape 2 — Sémantique de flux

delay_int. Pour $u = (u_0, u_1, \dots)$, on obtient :

$$y = (0, u_0, u_1, \dots).$$

toggle. Sans entrée utile (ou entrée ignorée), la sortie est :

$$y = (0, 1, 0, 1, \dots).$$

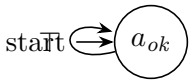
4.3 Étape 3 — Automates d'hypothèse et de garantie

delay_int. L'exemple concret `delay_int.kairos` porte la garantie

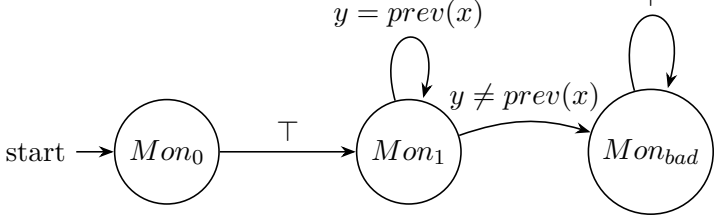
$$X\, G(y = prev(x)).$$

Dans l'implémentation, cette formule est compilée en un petit automate de résidus (trois états de moniteur) et le terme $prev(x)$ est matérialisé par une variable d'historique d'un tick. Le dessin ci-dessous suit cette lecture moniteur, plus fidèle au cas concret que la version précédente paramétrée par une valeur z.

delay_int — automate d'hypothèse A



delay_int — automate de garantie G

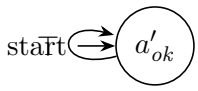


Ici :

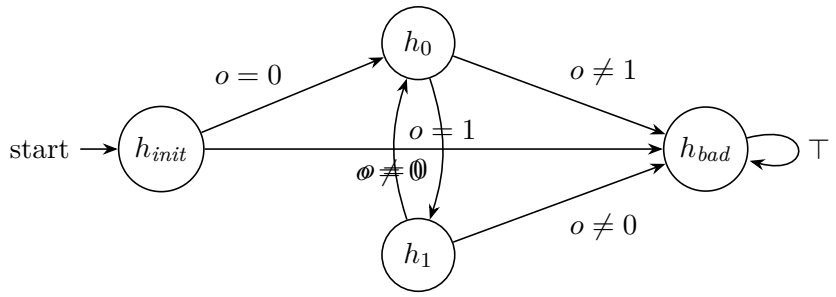
- Mon_0 représente le résidu initial avant que le **X** soit consommé ;
- Mon_1 représente le résidu stable $G(y = prev(x))$;
- Mon_{bad} représente la violation.

toggle. Pour `toggle`, le moniteur de garantie reste naturellement un automate fini sur la sortie observée.

toggle — automate d'hypothèse A



toggle — automate de garantie G



4.4 Étape 4 — Produit programme $\times A \times G$

État produit :

$$PS := S \times Q_A \times Q_G.$$

Un tick calcule un pas programme, puis fait avancer A avec $i(k)$ et G avec $(i(k), o(k))$.

delay_int.

$$(s_{init}, a_{ok}, g_{init}) \to (s_{run}, a_{ok}, Mon_1) \to \cdots$$

toggle.

$$(\tau_{init}, a_{ok}, h_{init}) \to (\tau_{run}, a_{ok}, h_0) \to (\tau_{run}, a_{ok}, h_1) \to \cdots$$

4.5 Étape 5 — Obligations locales par transition

L'idée n'est pas de générer une contrainte mystérieuse indexée par k ou $k+1$, mais une *contrainte locale sur un tick unique*. On choisit un pas produit dangereux

$$p : (s, q_A, q_G) \longrightarrow (s', q'_A, bad_G),$$

et on génère une formule qui affirme : *ce tick précis ne peut pas réaliser simultanément la transition programme et les arêtes automates qui mèneraient à bad_G.*

Autrement dit, l'obligation n'est pas une formule temporelle à part. C'est une formule du style :

$$\neg(\text{source courante correcte} \wedge \text{garde programme} \wedge \text{garde de } A \wedge \text{garde de } G).$$

Dans le backend, cette formule peut ensuite être projetée vers un `requires`, un `ensures` ou une combinaison des deux ; mais le noyau mathématique reste toujours : *interdire le matching d'un pas produit dangereux.*

delay_int. Le seul pas dangereux intéressant est celui qui part de

$$(s_{run}, a_{ok}, Mon_1)$$

et suit l'arête de garantie

$$Mon_1 \xrightarrow{y \neq prev(x)} Mon_{bad}.$$

La transition programme correspondante est la boucle

$$s_{run} \xrightarrow{\top \ / \ o := m,\ m' := i} s_{run}.$$

Donc le matching du pas dangereux est, conceptuellement :

$$\text{état programme } = s_{run} \ \wedge \ \text{état garantie } = Mon_1 \ \wedge \ o = m \ \wedge \ y \neq prev(x).$$

Comme ici $y = o$ et $prev(x) = m$ en régime établi, ce matching se réécrit comme $o = m \wedge o \neq m$, donc il doit être impossible. L'obligation locale générée exprime précisément cette impossibilité.

toggle. Il y a deux familles de pas dangereux :

— depuis h_0, une arête garantie vers h_{bad} étiquetée par $o \neq 1$;

— depuis h_1, une arête garantie vers h_{bad} étiquetée par $o \neq 0$.

Le programme, lui, exécute en boucle

$$\tau_{run} \xrightarrow{\top \ / \ o{:=}m,\ m'{:=}1-m} \tau_{run}.$$

Les obligations locales disent donc :

— si le moniteur est en h_0, le tick courant ne peut pas produire autre chose que 1 ;

— si le moniteur est en h_1, le tick courant ne peut pas produire autre chose que 0.

Là encore, ce sont des contraintes *sur le tick courant*, pas des formules temporelles sur plusieurs instants.

4.6 Étape 6 — Catégories d'obligations

La formalisation ne se contente pas d'énumérer des obligations. Elle distingue d'abord *pourquoi* une obligation existe, puis *où* elle est projetée dans le backend.

Quatre rôles abstraits. Au niveau logique abstrait, toute obligation générée tombe dans l'un des rôles suivants :

— `ObjectiveNoBad` : interdire un pas produit qui mènerait G vers bad_G ;

— `CoherencyGoal` : assurer que la représentation runtime du backend reste cohérente avec l'état abstrait du produit ;

— `SupportAutomaton` : fournir les contraintes auxiliaires nécessaires pour que les automates A et G soient exploités correctement dans le backend ;

— `SupportUserInvariant` : réinjecter les invariants ou contrats locaux fournis par l'utilisateur, indépendamment du produit.

Cette taxonomie est utilisée par les noyaux de correction pour séparer clairement :

— le coeur de sûreté (`ObjectiveNoBad`) ;

— la cohérence de l'encodage (`CoherencyGoal`) ;

— les hypothèses de support automates (`SupportAutomaton`) ;

— les invariants de support qui ne viennent pas du produit (`SupportUserInvariant`).

Opérateurs d'extraction intuitifs. On peut résumer la génération par quatre opérateurs :

— Extract_{obj} : prend un pas produit dangereux et construit l'obligation qui interdit son matching ;

— Extract_{coh} : prend un état ou un pas produit atteignable et construit les obligations qui lient l'état abstrait (s, q_A, q_G) à sa représentation backend ;

— Extract_{auto} : prend un état ou un pas atteignable et produit les hypothèses auxiliaires de compatibilité automate ;

— Extract_{user} : relaie les contrats ou invariants déjà attachés aux transitions du programme.

Autrement dit, toutes les obligations ne viennent pas de la même source, mais elles sont maintenant rangées dans une grille unique.

Couverture explicite de l'OBC augmenté. Au niveau pré-Why3 (formules injectées dans l'OBC/Kairos augmenté), la formalisation raffine ensuite ces rôles abstraits en familles plus fines :

— `FamTransitionRequires`, `FamTransitionEnsures`,

— `FamCoherencyRequires`, `FamCoherencyEnsuresShifted`,

— `FamInitialCoherencyGoal`,

— `FamNoBadRequires`, `FamNoBadEnsures`,

— `FamMonitorCompatibilityRequires`,

— `FamStateAwareAssumptionRequires`.

Le sens intuitif est le suivant :

— `FamNoBadRequires`/`FamNoBadEnsures` : projection de Extract_{obj} ;

— `FamCoherencyRequires`, `FamCoherencyEnsuresShifted`, `FamInitialCoherencyGoal` : projection de Extract_{coh} ;

— `FamMonitorCompatibilityRequires`, `FamStateAwareAssumptionRequires` : projection de Extract_{auto} ;

— `FamTransitionRequires`/`FamTransitionEnsures` : projection de Extract_{user}.

Cette couche est capturée par `obligations/ObcAugmentationSig.v`.

4.7 Étape 7 — Pont vers l'outil externe

Le coeur mathématique reste au niveau "triplets/obligations de transition". L'implémentation traduit ensuite ces triplets vers un format externe (Why3), possiblement avec encodage de l'état automate par variables de programme.

La preuve suppose uniquement :

$$\text{(validité externe de l'encodage)} \Rightarrow \text{(validité des triplets abstraits)}.$$

Donc `delay_int` et `toggle` partagent la même architecture de preuve, indépendamment du format concret envoyé à Why3.

4.8 Étape 8 — Théorème global

Pour chaque exemple :

$$\forall u,\ \text{AvoidA}(u) \Rightarrow \text{AvoidG}(\text{run}(u)).$$

delay_int. Sous hypothèse d'entrée admissible, la sortie respecte toujours $y = (0, u_0, u_1, \dots)$, donc G reste hors *bad*.

toggle. Le premier tick impose $y_0 = 0$, puis l'alternance est préservée à chaque tick ; l'automate garantie n'atteint jamais *bad*.

5 Modèle formel

À partir de cette section, on quitte la lecture guidée par exemples pour fixer le cadre mathématique général. Les mêmes notions que dans la vue d'ensemble sont redonnées, mais cette fois sous une forme stable et réutilisable dans les énoncés de preuve.

5.1 Flux et domaines

On fixe les types :

$$I \text{ (valeurs d'entrée)}, \quad O \text{ (valeurs de sortie)}, \quad M \text{ (mémoire)}, \quad S \text{ (état de contrôle)}.$$

Un flux est une fonction $\mathbb{N} \to X$, notée $\mathrm{Str}(X)$.

5.2 Automate de programme

Définition 5.1 (Transition de programme)**.** *Une transition de programme $t \in T$ est munie de :*

$$\mathrm{src}(t) \in S, \quad \mathrm{dst}(t) \in S, \quad \mathrm{guard}(t) : M \times I \to \{\mathsf{true}, \mathsf{false}\},$$

$$\mathrm{upd}(t) : M \times I \to M, \quad \mathrm{out}(t) : M \times I \to O.$$

Définition 5.2 (Activation et résultat)**.** *Pour $s \in S, m \in M, i \in I$:*

$$\mathrm{enabled}(t, s, m, i) \;:\Longleftrightarrow\; \mathrm{src}(t) = s \wedge \mathrm{guard}(t, m, i).$$

Le résultat de t sur (m, i) est :

$$\mathrm{res}(t, m, i) = \big(\mathrm{dst}(t), \mathrm{upd}(t, m, i), \mathrm{out}(t, m, i)\big).$$

Définition 5.3 (Sémantique déterministe du pas)**.** *On se donne un sélecteur déterministe*

$$\mathrm{prog_select} : S \times M \times I \to T$$

tel que $\mathrm{enabled}(\mathrm{prog_select}(s, m, i), s, m, i)$ *pour tout (s, m, i). Le pas du programme est alors*

$$\mathrm{step}(s, m, i) := \mathrm{res}(\mathrm{prog_select}(s, m, i), m, i).$$

5.3 Exemple : retard d'une entrée d'un pas

On illustre la définition précédente avec un programme qui produit en sortie l'entrée retardée d'un tick, en injectant explicitement la constante 0 au premier tick. On fixe $I = O = M = \mathbb{Z}$, deux états de contrôle $S = \{s_{init}, s_{run}\}$ et deux transitions $T = \{t_{init}, t_{run}\}$ définies par :

$$\begin{aligned}
\mathrm{src}(t_{init}) &= s_{init}, & \mathrm{dst}(t_{init}) &= s_{run}, & \mathrm{guard}(t_{init})(m, i) &= \mathsf{true},\\
\mathrm{upd}(t_{init}, m, i) &= i, & \mathrm{out}(t_{init}, m, i) &= 0, & &\\
\mathrm{src}(t_{run}) &= s_{run}, & \mathrm{dst}(t_{run}) &= s_{run}, & \mathrm{guard}(t_{run})(m, i) &= \mathsf{true},\\
\mathrm{upd}(t_{run}, m, i) &= i, & \mathrm{out}(t_{run}, m, i) &= m. & &
\end{aligned}$$

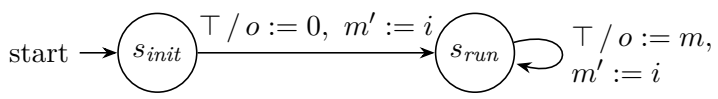
L'état initial de contrôle est $s_0 = s_{init}$ et la mémoire initiale $m_0 \in M$ est non déterminée. La première sortie vaut néanmoins toujours 0, car elle est fixée par $\mathrm{out}(t_{init}, m, i) = 0$ indépendamment de m. Si $u = (u_0, u_1, u_2, \dots)$, la sémantique de flux induit :

$$y(0) = 0, \qquad y(k+1) = u(k) \;\text{ pour tout } k \in \mathbb{N},$$

donc explicitement

$$y = (0, u_0, u_1, u_2, \dots).$$

L'automate de contrôle correspondant est :



Ce programme est déterministe : le premier tick exécute t_{init}, puis tous les ticks suivants exécutent t_{run}.

5.4 Exécution du programme

Soient $s_0 \in S$, $m_0 \in M$ les conditions initiales. Pour $u \in \mathrm{Str}(I)$, on définit récursivement :

$$\mathrm{cfg}(u, 0) = (s_0, m_0),$$

$$\mathrm{cfg}(u, k+1) = \pi_{S,M}\big(\mathrm{step}\big(\mathrm{cfg}(u, k), u(k)\big)\big)\,,$$

et la sortie au tick k :

$$o_u(k) = \pi_O\big(\mathrm{step}\big(\mathrm{cfg}(u, k), u(k)\big)\big)\,.$$

La trace observable est :

$$\mathrm{run}(u)(k) = \big(u(k), o_u(k)\big) \in I \times O.$$

6 Automates de sûreté

Cette section fixe la sémantique générale des automates A et G, d'abord comme automates déterministes sur mots infinis, puis sous la forme plus proche du code utilisée ensuite dans le produit.

6.1 Vue standard sur mots infinis

Définition 6.1 (Mot infini)**.** *Soit X un alphabet d'observation. Un mot infini sur X est une suite*

$$w = (w(0), w(1), w(2), \dots) \in \mathrm{Str}(X).$$

Dans ce document, un flux $\mathbb{N} \to X$ et un mot infini sur X sont donc le même objet mathématique.

Définition 6.2 (Automate de sûreté déterministe)**.** *Un automate de sûreté déterministe sur X est un quadruplet*

$$\mathcal{A} = (Q, q_0, bad, \delta),$$

où :

- *Q est un ensemble d'états ;*
- *$q_0 \in Q$ est l'état initial ;*
- *$bad \in Q$ est l'état de violation ;*
- *$\delta : Q \times X \to Q$ est la fonction de transition.*

Définition 6.3 (Run sur un mot infini)**.** *Étant donnés un automate $\mathcal{A} = (Q, q_0, bad, \delta)$ et un mot $w \in \mathrm{Str}(X)$, son exécution est la suite d'états*

$$\mathrm{run}_{\mathcal{A},w}(0) = q_0, \qquad \mathrm{run}_{\mathcal{A},w}(k+1) = \delta(\mathrm{run}_{\mathcal{A},w}(k), w(k)).$$

Autrement dit, l'état au tick $k+1$ se calcule à partir de l'état courant et de l'observation courante $w(k)$.

Définition 6.4 (Condition d'acceptation de sûreté)**.** *Un mot infini w est accepté par $\mathcal{A}$ si l'exécution n'atteint jamais l'état bad :*

$$\mathcal{A} \models_{safe} w \quad :\Longleftrightarrow \quad \forall k \in \mathbb{N},\ \mathrm{run}_{\mathcal{A},w}(k) \neq bad.$$

De manière équivalente, w est rejeté dès qu'il existe un préfixe fini sur lequel l'exécution atteint bad :

$$\neg(\mathcal{A} \models_{safe} w) \quad \Longleftrightarrow \quad \exists k \in \mathbb{N},\ \mathrm{run}_{\mathcal{A},w}(k) = bad.$$

Cette lecture est celle utilisée partout dans la formalisation Rocq :

- $\mathrm{AvoidA}(u)$ signifie que le mot d'entrée u est accepté par l'automate d'hypothèse A ;
- $\mathrm{AvoidG}(\mathrm{run}(u))$ signifie que le mot entrée/sortie $\mathrm{run}(u)$ est accepté par l'automate de garantie G.

6.2 Vue arêtes étiquetées

6.3 Des mots d'alphabet aux prédicats d'état et d'historique

Dans la présentation automates-théorie des langages, un automate lit un mot

$$w \in X^\omega,$$

où X est un alphabet. Ici, l'"alphabet" n'est pas forcément un ensemble de symboles atomiques déjà discrétisés : on prend comme observations des valuations ou des fragments d'état du système.

Définition 6.5 (Alphabet d'observation sémantique)**.** *On fixe un domaine d'observation X. Typiquement :*

— *pour l'hypothèse A, $X = I$ (valeur d'entrée courante) ;*

— *pour la garantie G, $X = I \times O$ (observation entrée/sortie du tick) ;*

— *plus généralement, X peut être un espace de valuations de variables observables.*

Un "symbole lu" au tick k est alors une observation $x_k \in X$.

Définition 6.6 (Contexte d'évaluation au tick)**.** *Pour interpréter des formules parlant du présent et du passé, on n'évalue pas les labels sur un symbole brut isolé, mais sur un contexte de tick contenant suffisamment d'information. On note ce contexte abstrait*

$$Ctx(k),$$

et il contient au minimum :

— *les valeurs courantes observables au tick k ;*

— *l'état/mémoire utiles à l'interprétation ;*

— *l'historique nécessaire pour donner un sens à des termes comme $prev(x)$ ou $pre_k(x, d)$.*

Dans la formalisation Rocq, ce rôle est joué par le type `StepCtx`*.*

Définition 6.7 (Atomes et labels sémantiques)**.** *Un atome observable est une formule α interprétée par un prédicat*

$$\mathrm{Sem}(\alpha) : Ctx \to \{\mathsf{true}, \mathsf{false}\}.$$

Un label d'arête est alors simplement un prédicat sur les contextes (ou sur les observations quand aucun historique n'est nécessaire).

Le passage "automate sur alphabet" → "automate sur états/formules" se fait donc en deux étapes :

1. on choisit comme alphabet X un domaine d'observations suffisamment riche (valeurs d'entrée, valeurs entrée/sortie, valuation partielle, etc.) ;
2. on interprète les labels comme des prédicats sémantiques sur le contexte du tick, plutôt que comme de simples comparaisons sur un symbole syntaxique atomique.

Cas des atomes avec passé. Quand une formule mentionne le passé, par exemple $prev(x)$ ou plus généralement $pre_k(x, d)$, le label ne peut pas être évalué à partir de la seule observation courante x_k. Il faut enrichir le contexte avec un historique. Formellement, on suppose une fonction d'évaluation

$$\mathrm{Eval}(Ctx(k), pre_k(x, d)) = x_{k-d}, \qquad \text{quand } k \geq d.$$

Ainsi, un label comme $y = prev(x)$ signifie en réalité :

$$\mathrm{Sem}(y = prev(x))(Ctx(k)) \quad :\Longleftrightarrow \quad y_k = x_{k-1}.$$

Dans l'implémentation, cet historique peut être réalisé de deux manières équivalentes pour la preuve :

— soit par une variable explicite d'historique, comme `__pre_k1_x` ;
— soit par une mémoire de programme reliée au passé par invariant, comme `z = prev(x)` dans `delay_int`.

Autrement dit, l'automate continue bien à reconnaître un langage de mots infinis, mais les "lettres" du mot sont des observations sémantiques évaluées sur des contextes de tick, et non des symboles purement syntaxiques sans mémoire.

Définition 6.8 (Automate de sûreté)**.** *Un automate de sûreté sur X est donné par :*

$$(Q, q_0, bad, E, \mathrm{src}_E, \mathrm{dst}_E, \lambda_E),$$

où $\lambda_E(e) : X \to \{\mathsf{true}, \mathsf{false}\}$ est un label (formule FO vue comme prédicat).

Définition 6.9 (Sélecteur d'arête)**.** *On suppose un sélecteur déterministe d'arête*

$$\mathrm{select} : Q \times X \to E$$

tel que, pour tout (q, x) :

$$\mathrm{src}_E(\mathrm{select}(q, x)) = q, \qquad \lambda_E(\mathrm{select}(q, x))(x) = \mathsf{true}.$$

Le pas d'automate est :

$$\delta(q, x) := \mathrm{dst}_E(\mathrm{select}(q, x)).$$

On instancie cela pour :
— l'automate d'hypothèse A sur I,
— l'automate de garantie G sur $I \times O$.

Exemple (retard d'un pas). Pour le programme de la sous-section précédente ($I = O = M = \mathbb{Z}$), on peut prendre :

$$Q_A = \{a_{ok}, a_{bad}\}, \quad q_0^A = a_{ok}, \quad bad_A = a_{bad}.$$

Avec les arêtes :

$$\lambda_A(e_{A,ok})(i) = \mathsf{true},\ \mathrm{src}_A(e_{A,ok}) = a_{ok},\ \mathrm{dst}_A(e_{A,ok}) = a_{ok},$$
$$\lambda_A(e_{A,bad})(i) = \mathsf{true},\ \mathrm{src}_A(e_{A,bad}) = a_{bad},\ \mathrm{dst}_A(e_{A,bad}) = a_{bad}.$$

L'automate d'entrée A ne contraint donc pas le flux d'entrée.

Pour la garantie `delay_int`, il est plus fidèle de prendre un automate de résidus pour la formule

$$X\, G(y = prev(x)).$$

On note :

$$Q_G = \{Mon_0, Mon_1, Mon_{bad}\}, \quad q_0^G = Mon_0, \quad bad_G = Mon_{bad}.$$

Lecture des états :
— Mon_0 : état initial, avant consommation du $\mathbf{X}$;
— Mon_1 : état stable où l'on vérifie à chaque tick $y = prev(x)$;
— Mon_{bad} : état absorbant de violation.

Les arêtes de G sont alors :

$$Mon_0 \xrightarrow{\mathsf{true}} Mon_1,$$
$$Mon_1 \xrightarrow{y=prev(x)} Mon_1, \qquad Mon_1 \xrightarrow{y\neq prev(x)} Mon_{bad},$$
$$Mon_{bad} \xrightarrow{\mathsf{true}} Mon_{bad}.$$

Dans l'implémentation, le terme $prev(x)$ n'est pas un état supplémentaire de l'automate : il est réalisé par la variable d'historique `__pre_k1_x` (ou, dans le noeud source, via la mémoire locale `z` et l'invariant $z = prev(x)$ en état `Run`). Ainsi G accepte exactement les traces qui satisfont, à partir du tick suivant l'initialisation, la contrainte $y = prev(x)$.

6.4 Sémantique des états automates sur flux

$$q_A(0) = q_0^A, \quad q_A(k+1) = \delta_A\big(q_A(k), u(k)\big).$$
$$q_G(0) = q_0^G, \quad q_G(k+1) = \delta_G\big(q_G(k), \mathrm{run}(u)(k)\big).$$

Notations de sûreté :

$$\mathrm{AvoidA}(u) \;:\Longleftrightarrow\; \forall k,\ q_A(k) \neq bad_A,$$
$$\mathrm{AvoidG}(\mathrm{run}(u)) \;:\Longleftrightarrow\; \forall k,\ q_G(k) \neq bad_G.$$

7 Produit programme $\times A \times G$

On peut maintenant combiner les trois dynamiques précédentes. Cette section est le coeur sémantique du document : elle explique ce qu'est un état du produit, ce qu'est un pas du produit, et comment un tick concret réalise un tel pas.

7.1 État et pas du produit

Intuition opérationnelle. On fixe un flux d'entrée

$$u = (i(0), i(1), i(2), \dots),$$

où $i(k)$ désigne l'entrée lue au tick k. Sous cette entrée, le programme produit une sortie observable $o(k)$ au tick k, ainsi qu'un état de contrôle et une mémoire mis à jour.

L'idée du produit programme $\times A \times G$ est de faire évoluer *en même temps* :

— l'état programme, en exécutant la transition choisie au tick k ;

— l'automate d'hypothèse A, en lui faisant lire l'observation d'entrée $i(k)$;

— l'automate de garantie G, en lui faisant lire l'observation entrée/sortie $(i(k), o(k))$.

Autrement dit, un tick ne calcule pas d'abord une exécution programme puis, dans un second temps indépendant, une exécution d'automate : il réalise un *pas produit* unique qui synchronise les trois composantes. Si l'on note

$$\mathrm{rps}(u,k) = \big(s_u(k), q_A(k), q_G(k)\big),$$

alors le tick k calcule directement le passage

$$\mathrm{rps}(u,k) \longrightarrow \mathrm{rps}(u,k+1),$$

en utilisant l'observation du tick courant, c'est-a-dire $i(k)$ pour A et $(i(k), o(k))$ pour G.

Ce que fait un tick k. Si, au début du tick k, le programme est dans l'état de contrôle $s_u(k)$ avec la mémoire $m_u(k)$, alors :

1. on lit l'entrée courante $i(k) = u(k)$;
2. le programme choisit sa transition

$$t = \mathrm{prog_select}(s_u(k), m_u(k), i(k));$$

3. cette transition calcule la sortie courante $o(k)$ et la configuration suivante $(s_u(k+1), m_u(k+1))$;
4. l'automate A avance de $q_A(k)$ vers $q_A(k+1)$ en lisant $i(k)$;
5. l'automate G avance de $q_G(k)$ vers $q_G(k+1)$ en lisant $(i(k), o(k))$.

Le produit sert précisément à rassembler ces trois évolutions dans un seul objet mathématique. C'est cette présentation qui remplace les notations plus opaques du style "$g_{run}(\cdot)$" : dans cette version du document, on ne paramètre plus un état de garantie par une valeur implicite. L'historique éventuel ($prev(x)$, mémoire locale, variable retardée) est porté soit par l'état programme, soit par le contexte d'évaluation, tandis que l'automate G reste un automate fini ordinaire avec des états nommés tels que Mon_0, Mon_1, Mon_{bad}.

Définition 7.1 (État produit)**.**

$$PS := S \times Q_A \times Q_G.$$

Au tick k, l'état produit concret est :

$$\mathrm{rps}(u, k) := (s_u(k), q_A(k), q_G(k)).$$

Exemple (automate produit du delay). Avec $S = \{s_{init}, s_{run}\}$, A et G ci-dessus :

$$\mathrm{rps}(u, 0) = (s_{init}, a_{ok}, Mon_0).$$

Si $u(0) = u_0$, le pas avec t_{init} (sortie 0) mène à :

$$\mathrm{rps}(u, 1) = (s_{run}, a_{ok}, Mon_1).$$

Puis, pour $k \geq 1$, si $\mathrm{rps}(u, k) = (s_{run}, a_{ok}, Mon_1)$, le pas avec t_{run} maintient la composante garantie dans Mon_1 tant que la contrainte $y = prev(x)$ est satisfaite :

$$\mathrm{rps}(u, k + 1) = (s_{run}, a_{ok}, Mon_1).$$

En cas de sortie non conforme, la composante garantie passe en Mon_{bad}, ce qui donne un pas produit dangereux.

Définition 7.2 (Pas produit candidat)**.** *Un pas produit p contient :*

$$p = (\mathrm{from}, t, e_A, e_G, m, i),$$

où :

— from $\in S \times Q_A \times Q_G$ *est l'état produit courant ;*

— t *est la transition programme exécutée au tick ;*

— e_A *est l'arête lue par A sur $i(k)$;*

— e_G *est l'arête lue par G sur $(i(k), o(k))$;*

— m *est la mémoire programme au début du tick ;*

— i *est l'entrée courante, c'est-à-dire $i(k)$.*

Définition 7.3 (Bien-formation)**.** *p est bien formé si :*

1. *t est activée sur la composante programme de* from *avec (m, i),*
2. *e_A est activée depuis la composante A de* from *sur i,*
3. *e_G est activée depuis la composante G de* from *sur $(i, \mathrm{out}(t, m, i))$,*
4. *les destinations de e_A, e_G coïncident avec δ_A, δ_G.*

Définition 7.4 (Cible dangereuse)**.** *Pour un pas p, sa cible est dite dangereuse si :*

$$q'_A \neq bad_A \quad \wedge \quad q'_G = bad_G.$$

7.2 Pas réalisé au tick k

Définition 7.5 (Réalisation)**.** *Un pas produit p est réalisé au tick k sous u si :*

— $\text{from}(p) = \text{rps}(u, k)$,

— *m est la mémoire concrète à k,*

— $i = u(k)$,

— $t = \text{prog_select}(s_u(k), m_u(k), u(k))$,

— *e_A est exactement l'arête suivie par A depuis $q_A(k)$ en lisant $i(k)$,*

— *e_G est exactement l'arête suivie par G depuis $q_G(k)$ en lisant $(i(k), o(k))$.*

8 Construction des obligations locales

Une fois le produit défini, les obligations locales ne sont plus des objets ajoutés de l'extérieur : elles sont construites à partir des pas produits dangereux. Cette section explicite cette construction et ses exemples.

8.1 Contexte local

Le contexte local au tick k contient :

$$(k,\ s_u(k),\ m_u(k),\ q_A(k),\ q_G(k),\ u(k),\ o_u(k),\ s_u(k+1),\ m_u(k+1)).$$

On note ce contexte $\text{ctx}(u, k)$. Il contient exactement les informations nécessaires pour savoir quel pas produit a été réalisé au tick k : source programme, mémoire courante, états automates courants, entrée, sortie et configuration programme suivante.

8.2 Définition formelle du matching

Définition 8.1 (Matching local d'un pas produit)**.** *Soit*

$$p = (\text{from}, t, e_A, e_G, m, i)$$

un pas produit bien formé, avec

$$\text{from} = (s, q_A, q_G).$$

On dit qu'un contexte local

$$c = (k, s_c, m_c, q_A^c, q_G^c, i_c, o_c, s'_c, m'_c)$$

matche *p, et on note* $\text{match}(c, p)$*, si les conditions suivantes sont toutes vraies :*

1. *la source produit coïncide :*

$$s_c = s, \qquad q_A^c = q_A, \qquad q_G^c = q_G;$$

2. *l'entrée et la mémoire courantes coïncident :*

$$m_c = m, \qquad i_c = i;$$

3. *la transition programme t est bien celle exécutée sur (s, m, i) ;*
4. *la sortie et la mémoire suivante observées dans c sont exactement celles calculées par t :*

$$(s'_c, m'_c, o_c) = \text{res}(t, m, i);$$

5. *l'arête e_A est bien l'arête d'hypothèse suivie depuis q_A sur i ;*
6. *l'arête e_G est bien l'arête de garantie suivie depuis q_G sur (i, o_c).*

Lecture intuitive. $\mathrm{match}(c, p)$ signifie : *le tick décrit par c est exactement le tick qui réalise le pas produit p.* Ce n'est donc pas une condition abstraite supplémentaire ; c'est l'égalité entre :

— un tick concret de l'exécution ;

— un pas produit candidat décrit dans le produit *programme* $\times A \times G$.

8.3 Construction formelle de l'obligation

Définition 8.2 (Obligation d'interdiction d'un pas)**.** *À tout pas produit bien formé p, on associe la formule locale*

$$\mathrm{obl}_p(c) := \neg\, \mathrm{match}(c, p).$$

Autrement dit, obl_p *affirme que le tick courant* n'est pas *une réalisation du pas p.*

Définition 8.3 (Pas générateurs)**.** *On s'intéresse prioritairement aux pas produits dont la cible est dangereuse :*

$$p : (s, q_A, q_G) \longrightarrow (s', q'_A, bad_G)$$
$$\textit{avec } q'_A \neq bad_A.$$

Ce sont ces pas qui doivent être interdits par les obligations locales utilisées dans la preuve de correction.

Forme concrète. Dans tous les exemples du document, $\mathrm{match}(c, p)$ se développe en une conjonction finie de contraintes locales :

$$\text{source produit} \wedge \text{garde programme} \wedge \text{garde } A \wedge \text{garde } G.$$

L'obligation générée est simplement la négation de cette conjonction. C'est pourquoi, dans l'implémentation, on la retrouve souvent sous une forme proche de

$$\neg(\mathrm{guard}_{prog} \wedge \mathrm{guard}_A \wedge \mathrm{guard}_G),$$

éventuellement enrichie par la composante "on est bien dans le bon état produit".

8.4 Génération relationnelle

Définition 8.4 (Relation GeneratedBy)**.** $\mathrm{GeneratedBy}(o, t, \varphi)$ *vaut s'il existe un pas produit bien formé p tel que :*

$$t = \mathrm{tr}(p), \qquad \varphi = \mathrm{obl}_p, \qquad o = \mathrm{classify}(p).$$

On pose :

$$\mathrm{Generated}(\varphi) :\Longleftrightarrow \exists o, t,\ \mathrm{GeneratedBy}(o, t, \varphi).$$

8.5 Classification et opérateurs d'extraction

Raffinement de l'origine des obligations. Pour rendre explicite d'où viennent les obligations, on peut raffiner l'origine abstraite o en constructeurs du type :

$$\begin{aligned} o ::= {} & \mathrm{ObjStep}(p) \mid \mathrm{CohSrc}(x) \mid \mathrm{CohDst}(p) \mid \mathrm{CohInit} \\ & \mid \mathrm{AutoCompat}(x) \mid \mathrm{AutoAssume}(p) \mid \mathrm{UserReq}(t, \psi) \mid \mathrm{UserEns}(t, \psi), \end{aligned}$$

où x désigne un état produit atteignable, p un pas produit atteignable, et ψ une formule locale déjà fournie par le programme.

Extraction objective. L'extraction objective part d'un pas produit dangereux

$$p : (s, q_A, q_G) \to (s', q'_A, bad_G) \qquad \text{avec } q'_A \neq bad_A,$$

et définit

$$\mathrm{Extract}_{obj}(p) := \mathrm{obl}_p.$$

Intuitivement : *ce pas ne doit jamais matcher*. C'est la source abstraite de la catégorie `ObjectiveNoBad` et, après projection backend, des familles `FamNoBadRequires`/`FamNoBadEnsures`.

Extraction de cohérence. L'exécution concrète encode l'état abstrait du produit via une représentation runtime (par exemple `__aut_state`). Il faut donc des obligations de cohérence pour relier les deux vues. On peut les décrire abstraitement par :

— une obligation initiale

$$\mathrm{coh}_{init} : \text{“au tick 0, l'encodage runtime représente bien l'état initial du produit”};$$

— pour chaque état produit atteignable $x = (s, q_A, q_G)$, une obligation de source

$$\mathrm{coh}^{src}_x : \text{“si le backend prétend être en } x\text{, alors la source abstraite courante est bien } x\text{”};$$

— pour chaque pas atteint

$$p : x \to x',$$

une obligation de cible

$$\mathrm{coh}^{dst}_p : \text{“si le tick réalise } p\text{, alors l'encodage runtime au tick suivant représente } x'\text{”}.$$

On pose alors :

$$\mathrm{Extract}_{coh}(x) = \{\mathrm{coh}^{src}_x\}, \qquad \mathrm{Extract}_{coh}(p) = \{\mathrm{coh}^{dst}_p\},$$

plus l'obligation initiale coh_{init}. Cette extraction alimente abstraitement `CoherencyGoal`, puis, après projection backend, les familles `FamCoherencyRequires`, `FamCoherencyEnsuresShifted`, `FamInitialCoherencyGoal`.

Extraction de support automate. Le backend a aussi besoin d'obligations qui ne sont pas des objectifs de sûreté, mais qui rendent l'exploitation des automates correcte. On distingue typiquement :

— pour un état produit atteignable x, une obligation de compatibilité

$$\mathrm{compat}_x : \text{“l'encodage runtime autorise exactement les combinaisons cohérentes avec } x\text{”};$$

— pour un pas p, une obligation d'hypothèse guidée par l'état

$$\mathrm{assume}_p : \text{“si l'on est dans la source de } p\text{, l'entrée courante respecte bien l'arête d'hypothèse admissible”}.$$

On pose :

$$\mathrm{Extract}_{auto}(x) = \{\mathrm{compat}_x\}, \qquad \mathrm{Extract}_{auto}(p) = \{\mathrm{assume}_p\}.$$

Cette extraction nourrit abstraitement `SupportAutomaton`, puis, après projection backend, `FamMonitorCompati` et `FamStateAwareAssumptionRequires`.

Extraction de support utilisateur. Certaines obligations ne viennent pas du produit, mais sont déjà attachées au programme sous forme de préconditions, postconditions ou invariants locaux. Si une transition t porte une obligation locale ψ, on définit :

$$\mathrm{Extract}_{user}(t, \psi) = \{\psi\}.$$

Cette extraction alimente `SupportUserInvariant` ; après projection backend, on la retrouve dans `FamTransitionRequires` ou `FamTransitionEnsures`.

Résumé formel. La génération complète peut se lire comme l'union des images de ces opérateurs :

$$\mathrm{Generated} \;=\; \mathrm{Im}(\mathrm{Extract}_{obj}) \;\cup\; \mathrm{Im}(\mathrm{Extract}_{coh}) \;\cup\; \mathrm{Im}(\mathrm{Extract}_{auto}) \;\cup\; \mathrm{Im}(\mathrm{Extract}_{user}).$$

Le rôle des taxonomies Rocq est alors :

— de partitionner cette union selon le *rôle logique* des obligations ;

— puis de raffiner encore cette partition selon la *famille backend* utilisée dans l'OBC augmenté.

8.6 Exemples détaillés

Exemple complet : `delay_int`, premier tick sûr. Le premier tick part de

$$\mathrm{rps}(u, 0) = (s_{init}, a_{ok}, Mon_0)$$

et exécute la transition programme

$$t_{init} : \; s_{init} \to s_{run}, \qquad o := 0, \qquad m' := i.$$

Dans le même tick :

— A reste en a_{ok} sur $\top$;

— G passe de Mon_0 à Mon_1 sur l'arête $\top$.

Le pas produit correspondant est sûr, car sa cible est

$$(s_{run}, a_{ok}, Mon_1),$$

et non un état avec bad_G. Il n'engendre donc pas l'obligation critique de sûreté ; il sert seulement à illustrer le mécanisme synchronisé du produit.

Exemple complet : `delay_int`, pas dangereux en régime établi. Supposons maintenant $k \geq 1$ et

$$\mathrm{rps}(u, k) = (s_{run}, a_{ok}, Mon_1).$$

Les automates graphiques déjà dessinés plus haut imposent alors :

— programme :

$$s_{run} \xrightarrow{\top \;/\; o{:=}m,\; m'{:=}i} s_{run};$$

— hypothèse :

$$a_{ok} \xrightarrow{\top} a_{ok};$$

— garantie dangereuse :

$$Mon_1 \xrightarrow{y \neq prev(x)} Mon_{bad}.$$

Le matching de ce pas dangereux se lit donc exactement comme :

$$\mathrm{match}(c,p) \equiv (s = s_{run}) \wedge (q_A = a_{ok}) \wedge (q_G = Mon_1) \wedge (o = m) \wedge (y \neq prev(x)).$$

Dans cet exemple, $y = o$ et, en régime établi, $prev(x) = m$. Donc :

$$\mathrm{match}(c,p) \equiv (s = s_{run}) \wedge (q_A = a_{ok}) \wedge (q_G = Mon_1) \wedge (o = m) \wedge (o \neq m).$$

L'obligation générée est alors :

$$\mathrm{obl}_p(c) \equiv \neg\Big((s = s_{run}) \wedge (q_A = a_{ok}) \wedge (q_G = Mon_1) \wedge (o = m) \wedge (o \neq m)\Big).$$

Le backend peut projeter cette formule vers une forme plus simple et plus directement prouvable, par exemple une obligation équivalente du style

$$q_G = Mon_1 \Rightarrow y = prev(x).$$

Mais la source mathématique est bien l'interdiction du pas produit dangereux.

Exemple complet : `toggle`, branche dangereuse depuis h_0. Considérons un tick où

$$\mathrm{rps}(u,k) = (\tau_{run}, a'_{ok}, h_0).$$

Les automates graphiques imposent :

— programme :

$$\tau_{run} \xrightarrow{\top \ / \ o:=m,\ m':=1-m} \tau_{run};$$

— hypothèse :

$$a'_{ok} \xrightarrow{\top} a'_{ok};$$

— garantie dangereuse :

$$h_0 \xrightarrow{o \neq 1} h_{bad}.$$

Le matching du pas dangereux est donc :

$$\mathrm{match}(c,p) \equiv (s = \tau_{run}) \wedge (q_A = a'_{ok}) \wedge (q_G = h_0) \wedge (o = m) \wedge (o \neq 1).$$

L'obligation générée interdit exactement cette conjonction :

$$\mathrm{obl}_p(c) \equiv \neg\Big((s = \tau_{run}) \wedge (q_A = a'_{ok}) \wedge (q_G = h_0) \wedge (o = m) \wedge (o \neq 1)\Big).$$

Après projection backend, on retrouve une obligation plus lisible du style

$$q_G = h_0 \Rightarrow o = 1.$$

Exemple complet : `toggle`, branche dangereuse depuis h_1. De façon symétrique, si

$$\mathrm{rps}(u,k) = (\tau_{run}, a'_{ok}, h_1),$$

alors l'arête dangereuse est

$$h_1 \xrightarrow{o \neq 0} h_{bad},$$

et l'obligation générée interdit :

$$(s = \tau_{run}) \wedge (q_A = a'_{ok}) \wedge (q_G = h_1) \wedge (o = m) \wedge (o \neq 0).$$

Le backend peut la projeter sous la forme équivalente

$$q_G = h_1 \Rightarrow o = 0.$$

Conclusion des exemples. Dans tous les cas, on construit l'obligation de la même manière :

1. choisir un pas produit dont la cible met G dans bad_G ;
2. développer le *matching* de ce pas comme conjonction de contraintes locales sur le tick courant ;
3. prendre la négation de cette conjonction.

Les indices k, $k+1$ servent uniquement à dire *sur quel tick concret* on lit la source et la cible du pas. Ils ne veulent pas dire que l'obligation elle-même est une formule temporelle à deux instants.

8.7 Oracle

On suppose un oracle booléen Oracle sur les obligations, avec :

— **Soundness** :

$$\text{Oracle}(\varphi) = \mathsf{true} \Rightarrow \varphi \text{ est valide sur tous ticks/toutes exécutions.}$$

— **Completeness** :

$$\text{Generated}(\varphi) \Rightarrow \text{Oracle}(\varphi) = \mathsf{true}.$$

9 Chaîne de preuve

Les lemmes de cette section se lisent dans l'ordre logique suivant :

$$\neg\text{AvoidG} \Rightarrow \exists \text{ pas dangereux local} \Rightarrow \exists \text{ obligation générée violée.}$$

Le théorème principal ajoute ensuite les hypothèses oracle pour remonter de cette chaîne locale à la correction globale.

9.1 Lemmes de base

Lemme 9.1 (Existence d'un pas réalisé)**.** *Pour tout u, k, il existe un pas produit bien formé réalisé au tick k.*

Démonstration. On construit explicitement le pas à partir de :

— la transition prog_select$(s_u(k), m_u(k), u(k))$,

— les arêtes select$_A(q_A(k), u(k))$, select$_G(q_G(k), \text{run}(u)(k))$.

Les hypothèses de source/label des sélecteurs garantissent la bien-formation. □

Lemme 9.2 (Correction de cible)**.** *Si un pas produit bien formé p est réalisé au tick k, alors sa cible coïncide avec $(q_A(k+1), q_G(k+1))$ sur les composantes automates.*

Démonstration. Par développement de δ_A, δ_G, des sélecteurs, et de la définition de "réalisé". □

9.2 Du global au local

Définition 9.3 (Pas dangereux local)**.** BadLocal(u, k) *s'il existe un pas produit bien formé réalisé au tick k, dont la cible est dangereuse ($q'_A \neq bad_A$, $q'_G = bad_G$).*

Théorème 9.4 (Violation globale $\Rightarrow$ pas dangereux local)**.** *Si* AvoidA(u) *et* $\neg$AvoidG(run(u))*, alors*

$$\exists k,\ \text{BadLocal}(u, k).$$

Démonstration. On prend le premier indice j tel que $q_G(j) = bad_G$ (existence par $\neg$AvoidG). Le cas $j = 0$ est impossible ($q_0^G \neq bad_G$). Donc $j = S(k)$. Par le lemme d'existence d'un pas réalisé à k, puis correction de cible, la composante G de la cible vaut $q_G(S(k)) = bad_G$, et la composante A reste hors bad_A par AvoidA(u). Donc BadLocal(u, k). □

9.3 Couverture par obligations

Théorème 9.5 (Couverture locale)**.** *Si* $\mathrm{BadLocal}(u, k)$*, alors il existe une obligation générée* φ *telle que*

$$\neg\varphi(\mathrm{ctx}(u, k)).$$

Démonstration. Prendre le pas produit dangereux p qui témoigne $\mathrm{BadLocal}(u, k)$. Par définition de GeneratedBy, obl_p est générée. Comme p est réalisé à k, le contexte $\mathrm{ctx}(u, k)$ matche p, donc $\mathrm{obl}_p(\mathrm{ctx}(u, k))$ est faux. □

10 Théorème principal

Théorème 10.1 (Correction conditionnelle)**.** *Pour tout flux d'entrée* u *:*

$$\mathrm{AvoidA}(u) \Rightarrow \mathrm{AvoidG}(\mathrm{run}(u)).$$

Démonstration. Supposons $\mathrm{AvoidA}(u)$ et $\neg\mathrm{AvoidG}(\mathrm{run}(u))$. Par le théorème "violation globale $\Rightarrow$ pas dangereux local", il existe k avec $\mathrm{BadLocal}(u, k)$. Par couverture locale, il existe une obligation générée φ telle que $\neg\varphi(\mathrm{ctx}(u, k))$.

Par complétude oracle, $\mathrm{Oracle}(\varphi) = \mathsf{true}$. Par soundness oracle, φ est valide sur tous ticks, en particulier au tick k : $\varphi(\mathrm{ctx}(u, k))$, contradiction.

Donc $\mathrm{AvoidG}(\mathrm{run}(u))$. □

11 Lien avec Rocq et l'implémentation actuelle

Cette dernière partie ne rajoute pas de nouvelle sémantique. Elle explique comment les objets définis plus haut se retrouvent dans les fichiers Rocq et dans le pipeline OCaml, et où se situent encore les écarts d'architecture.

Cette section aligne explicitement le texte mathématique avec l'architecture modulaire en Rocq (version courante du dépôt).

11.1 Signatures centrales

- `core/CoreStepSig.v` (`CORE_STEP_SIG`) : sémantique réactive de base (cfg, ctx, run).
- `obligations/ObligationGenSig.v` (`OBLIGATION_GEN_SIG`) : obligations abstraites `StepCtx` → `Prop`, plus type abstrait `Origin` et relation `GeneratedBy`.
- `obligations/ObligationTaxonomySig.v` : catégories `ObjectiveNoBad`, `CoherencyGoal`, `SupportAutomato` `SupportUserInvariant` et partition de `Generated`.
- `obligations/ObligationStratifiedSig.v` : stratification des phases, avec notamment `objective_before` et `coherency_before_support_automaton`.
- `obligations/ObcAugmentationSig.v` : partition fine des formules de l'OBC augmenté avant validation externe, avec les familles `FamNoBad*`, `FamCoherency*`, `FamMonitor*`, `FamStateAware*`, `FamTransition*`.
- `obligations/OracleSemSig.v` (`ORACLE_SEM_SIG`) : interface oracle avec propriété pointwise `obligation_valid_pointwise`.

11.2 Noyaux de correction

- `KairosOracle.v` : preuve monolithique de référence avec progression explicite du produit via `product_select_at`, `realizable_product_step`, `product_progresses_at_each_tick`, puis noyau de correction via `bad_local_step_if_G_violated` et `generation_coverage`.

— `core/AutomataCorrectnessCore.v` : alias du noyau prouvé interne pour éviter toute re-axiomatisation.

— `integration/ThreeLayerFromCore.v` : reconstruction du lemme de couverture `coverage_if_not_avoidG` à partir des deux ingrédients prouvés "violation globale ⇒ pas local dangereux" et "pas local dangereux ⇒ obligation générée violée".

— `interfaces/ExternalValidationAssumptions.v` : frontière unique des hypothèses externes (`oracle_sound_true`, `oracle_complete_generated`).

— `integration/AutomataFinalCorrectness.v` : théorème final modulaire `automata_program_correctness`.

— `kernels/ObjectiveSafetyKernel.v` : `oracle_conditional_correctness_from_objectives` et version enrichie `oracle_conditional_correctness_with_coherency`.

— `kernels/SupportNonBlockingKernel.v` : préservation de correction sous changement d'oracle, y compris `correction_preserved_if_oracles_agree_on_objectives_and_coherency`.

— `integration/ThreeLayerArchitecture.v` : théorème trois couches `oracle_conditional_correctness_th`

11.3 Ponts avec l'implémentation externe

11.3.1 Produit explicite dans la preuve, instrumentation fragmentée dans le code

La formalisation Rocq prend comme sémantique de référence un produit explicite

$$PS = S \times Q_A \times Q_G,$$

avec un pas local qui transporte simultanément :

— la transition programme choisie au tick courant ;

— l'arête d'hypothèse A effectivement lue sur l'observation d'entrée ;

— l'arête de garantie G effectivement lue sur l'observation entrée/sortie.

La version actuelle de l'implémentation OCaml contient désormais un noyau produit explicite dans le middle-end. Ce noyau construit et explore des états (s, q_A, q_G) atteignables, calcule les pas synchronisés du triplet, les classe (*safe*, *bad_A*, *bad_G*, *pruned*) et alimente :

— la vue texte du produit ;

— les raisons de pruning ;

— une carte locale des obligations issues des pas *bad_G* ;

— la génération des hypothèses de compatibilité et des obligations locales projetées sur le backend instrumenté.

Concrètement, cette couche est portée par les modules `product_types`, `product_build`, `product_debug` et `product_contracts` du middle-end.

L'exécution concrète du backend reste toutefois instrumentée via un état runtime `__aut_state`. Le produit explicite joue donc le rôle de noyau sémantique et de diagnostic, tandis que l'instrumentation OBC/Why3 reste le mécanisme de compilation exécutable.

11.3.2 Pourquoi un produit explicite serait meilleur

Cette évolution est préférable pour quatre raisons.

1. Source unique de sémantique. Au lieu de répartir entièrement la logique de correction entre génération d'automates, invariants de compatibilité, instrumentation OBC et encodage Why3, le produit explicite fait porter la sémantique sur un seul objet : états, pas, classification des cibles, raisons de pruning.

2. Alignement direct avec Rocq. La preuve Rocq parle déjà de `ProductState`, `ProductStep`, `product_select_at`, `product_progresses_at_each_tick` et `GeneratedBy`. Le noyau produit côté OCaml crée une correspondance presque nom à nom entre preuve et implémentation, ce qui réduit le risque de dérive architecturale.

3. Obligation generation plus locale et plus traçable. Au lieu de reconstituer indirectement les obligations depuis plusieurs passes, on pourrait attacher chaque obligation à un pas produit dangereux précis, avec sa provenance $(s, q_A, q_G) \to (s', q'_A, q'_G)$, ses labels, et sa raison d'émission.

Le gain n'est donc pas principalement "faire plus de produit" au sens combinatoire, mais *rendre explicite le noyau sémantique déjà présent implicitement.*

4. Unification de la génération de contrats. La couche produit n'est plus seulement un outil de diagnostic : elle sert maintenant à dériver les invariants de compatibilité projetés sur `__aut_state`, les préconditions liées aux hypothèses, et les obligations locales bloquant les pas *bad_G*. On supprime ainsi une partie de la duplication antérieure entre "produit implicite" et génération de contrats.

11.3.3 Pourquoi c'est équivalent à l'implémentation actuelle

L'introduction du produit explicite ne change pas la sémantique visée ; elle change la représentation intermédiaire principale.

Vue actuelle. Le pipeline concret courant :

1. compile les contrats en automates/résidus ;
2. explore le produit programme $\times A \times G$ de manière finie et *on-the-fly* à partir de l'état initial ;
3. dérive depuis ce produit les diagnostics, les raisons de pruning, les hypothèses de compatibilité projetées, et les obligations locales associées aux pas *bad_G* ;
4. projette ensuite ces informations vers le moniteur principal `__aut_state` ;
5. émet enfin le backend Why3/OBC et les tâches externes associées.

Vue sémantique canonique. Le point de vue canonique est désormais :

1. construire l'état courant (s, q_A, q_G) ;
2. calculer le pas réalisé
$$(s, q_A, q_G) \xrightarrow{t, e_A, e_G} (s', q'_A, q'_G);$$
3. classer ce pas comme sûr, hypothèse-violée, garantie-violée ou pruné ;
4. projeter ce pas vers les diagnostics produit, les obligations locales, puis l'instrumentation et le backend Why3/OBC.

Équivalence sémantique attendue. Les deux lectures sont équivalentes si elles préservent simultanément :

— les mêmes triples atteignables (s, q_A, q_G) ;

— le même critère d'atteinte de bad_G ;

— les mêmes cas où A rend une combinaison non admissible ;

— la même famille d'obligations générées, modulo traduction syntaxique vers OBC/Why3 et projection sur `__aut_state`.

Autrement dit, le produit explicite n'ajoute pas de nouveaux comportements : il rassemble en un IR unique les comportements auparavant encodés de manière plus dispersée dans le pipeline. La relation correcte à viser est donc une *équivalence par projection/compilation* :

— tout pas instrumenté actuel correspond à un pas du produit explicite ;

— tout pas réalisé du produit explicite se recompile en mise à jour de moniteur, préconditions et obligations dans l'implémentation actuelle ;

— la couche A × G n'est plus un second noyau autonome, mais une projection du produit complet.

Dans cette lecture, l'architecture actuelle devient effectivement le backend d'une représentation produit plus canonique, et non une sémantique concurrente.

— `obligations/ImplementationValidatorBridge.v` : pont "validateur d'obligations" → `ORACLE_SEM_SIG`.

— `obligations/HoareExternalBridge.v` : pont "obligations → tâches/triplets Hoare → outil externe".

— `obligations/TransitionTriplesBridge.v` : pont "triplets par transition" + encodage externe abstrait (incluant l'hypothèse de soundness de l'encodage), via `MakeOracleSemFromTransitionTriple`

11.4 Blueprint de refactoring

Le fichier `KairosRefactorBlueprint.v` regroupe les alias de signatures et les foncteurs canoniques. Il sert de contrat d'architecture entre :

1. la couche sémantique programme/automates,
2. la couche de génération et classification d'obligations,
3. la couche de validation externe (sans figer Why3 dans le coeur formel).

12 Remarques de portée

Remarque 12.1. *Cette formalisation est indépendante d'une syntaxe LTL explicite : les entrées/sorties sont spécifiées directement par automates de sûreté.*

Remarque 12.2. *Les hypothèses externes restantes sont :*

— *validité des sélecteurs (*prog_select, select_A, select_G*),*

— *états initiaux non mauvais ($q_0^A \neq bad_A$, $q_0^G \neq bad_G$),*

— *soundness/completeness de l'oracle sur les obligations générées.*

Remarque 12.3. *Le point important de l'état courant est que la progression du produit n'est plus une hypothèse implicite : elle est construite explicitement, tick par tick, dans `KairosOracle.v`. Le théorème final modulaire n'assume donc que la validation externe des obligations générées, et non le non-blocage du produit.*